



Automata Learning from Preference and Equivalence Queries

Eric Hsiung^(✉), Joydeep Biswas, and Swarat Chaudhuri

University of Texas at Austin, Austin, TX 78712, USA
 {ehsiung, joydeepb, swarat}@cs.utexas.edu

Abstract. Active automata learning from membership and equivalence queries is a foundational problem with numerous applications. We propose a novel variant of the active automata learning problem: actively learn finite automata using *preference queries*—i.e., queries about the relative position of two sequences in a total preorder—instead of membership queries. Our solution is REMAP, a novel algorithm which leverages a symbolic observation table along with unification and constraint solving to navigate a space of symbolic hypotheses (each representing a set of automata), and uses satisfiability-solving to construct a concrete automaton (specifically a Moore machine) from a symbolic hypothesis. REMAP is guaranteed to correctly infer the minimal automaton with polynomial query complexity under exact equivalence queries, and achieves PAC-identification (ε -approximate, with high probability) of the minimal automaton using sampling-based equivalence queries. Our empirical evaluations of REMAP on the task of learning reward machines for two reinforcement learning domains indicate REMAP scales to large automata and is effective at learning correct automata from consistent teachers, under both exact and sampling-based equivalence queries.

1 Introduction

Active automata learning has applications from software engineering [1, 37] and verification [29] to interpretable machine learning [44] and learning reward machines [16, 20, 39, 46]. The classical problem formulation involves a teacher with access to a regular language and a learner which asks *membership* and *equivalence* queries to infer a finite automaton describing the regular language [5, 22].

Consider an alternative formulation: *learning a finite automaton from preference and equivalence queries*.¹ A preference query resolves the relative position of two sequences in a total preorder available to the teacher. The motivation for learning from preferences stems from leveraging human preferences as a rich source of information. In fact, comparative feedback, such as preferences, is a modality which humans are apt at providing in comparison to giving specific

¹ Shah et al. [38] investigates choosing between membership and preference queries.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-98685-7_5.

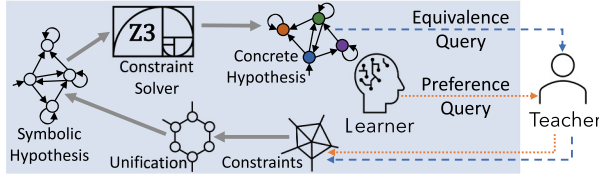


Fig. 1. REMAP Algorithm Overview.

numerical values, as shown by MacGlashan et al. [30] and Christiano et al. [12], likely due to *choice overload* [26]. But preferences need not be directly obtained from individual humans: preferences can also be obtained from automated systems in a frequency-based sense. For example, consider an obstacle avoidance scenario of a vehicle avoiding large debris in the middle of a roadway. Consider a dataset of a population of driver responses (vehicle trajectories) to such a scenario, procured from dashcam or traffic camera footage. The population preference of a given driving response can be automatically determined by ranking each response by the frequency of occurrence in the dataset. In fact, learning from human preference data has applications in fine-tuning language models [34], learning conditional preference networks [23, 28], inferring reinforcement learning policies [13], and learning Markovian reward functions [10, 11, 27, 36]. Possible applications for learning finite automata from preferences over sequences include inferring sequence classifications (e.g. program executions [21], vehicle maneuvers, human-robot interactions) using ordered classes (e.g. *safe*, *risky*, *dangerous*, *fatal*) with each automaton state labeled by a class, distilling interpretable preference models [44], and inferring reward machines. However, no method currently exists for learning finite automata from preferences with the termination and correctness guarantees enjoyed by classical automata learning algorithms such as L^* [5].

Unfortunately, adapting L^* to the preference-based setting is challenging. Preference queries do not directly provide the concrete observations available from membership queries, as required by L^* , so our solution REMAP addresses this challenge through a *symbolic approach* to L^* .

REMAP² (Fig. 1) features termination and correctness guarantees for *exact* and *probably approximately correct* [43] (PAC) *identification* [6] of the desired automaton, with a strong learner capable of symbolic reasoning and constraint-solving to offset the weaker preference-based signals from the teacher. While approximate learners have been proposed to learn from a combination of membership and preference queries [38], REMAP is the first exact learner with formal guarantees of minimality, correctness, and query complexity. By using unification³ to navigate the symbolic space of hypotheses, and constraint-solving

² Code available at <https://eric-hsiung.github.io/remap/>, as well as supplementary material.

³ Symbolically obtaining sets of equivalent variables from sets of equations, and substituting variables in the equations with their representatives. See Sect. 4.2.

to construct a concrete automaton from a symbolic hypothesis, REMAP identifies, in a polynomial number of queries, the minimal Moore machine isomorphic to one describing the teacher’s total preorder over input sequences when using *exact* equivalence queries. However, exact equivalence queries may be infeasible if the learner and teacher lack a common representation.⁴ This motivates *sampling-based* equivalence queries under REMAP, which achieves *PAC-identification*.⁵ Our empirical evaluations apply REMAP to learn reward machines for sequential-decision making domains in the reward machine literature. We measure query complexity for the exact and PAC-identification settings, and measure empirical correctness for the PAC-identification setting.

We contribute (a) REMAP, a novel L^* style algorithm for learning Moore machines using preference and equivalence queries; under *exact* and *PAC-identification* settings we provide (b) theoretical analysis of query complexity, correctness, and minimalism, and (c) supporting empirical results, demonstrating the efficacy of the algorithm’s ability to learn reward machines from preference and equivalence queries.

2 Background

Prior to introducing REMAP, we provide preliminaries on orders and finite automata, followed by a discussion of Angluin’s L^* algorithm in order to highlight how REMAP differs from L^* .

Definition 1 (Total Preorder). *The binary relation $\preceq$ on a set A is a total preorder if $\preceq$ satisfies: $a \preceq a$ for all $a \in A$ (reflexivity); $a \preceq b \wedge b \preceq c \implies a \preceq c$ for all $a, b, c \in A$ (transitivity); and $a \preceq b \vee b \preceq a$ for all $a, b \in A$ (total).*

Definition 2 (Total Order). *The binary relation $\preceq$ on a set A is a total order if $\preceq$ satisfies: total preorder conditions; and $a \preceq b \wedge b \preceq a \implies a = b$ for all $a, b \in A$ (antisymmetric); $a = b$ means a and b are the same element from A .*

Finite Automata. Automata describe sets of sequences. An *alphabet* Σ is a set whose elements can be used to construct sequences; Σ^* represents the set of sequences of any length created from elements of Σ . A sequence $s \in \Sigma^*$ has integer length $|s| \geq 0$; if $|s| = 0$, then s is the empty sequence ε . An element $\sigma \in \Sigma$ has length 1; if $s, t \in \Sigma^*$, then $s \cdot t$ represents s concatenated with t , with length $|s| + |t|$. Different types of finite automata have different semantics. Deterministic automata feature a deterministic transition function δ defined over a set of states Q and an input alphabet Σ^I . An output alphabet Σ^O may be present to label states or transitions using a labeling function L . The tuple $\langle Q, q_0, \Sigma^I, \Sigma^O, \delta, L \rangle$ is a *Moore machine* [33], where $q_0 \in Q$ is the initial state, $\delta : Q \times \Sigma^I \rightarrow Q$ describes transitions, and $L : Q \rightarrow \Sigma^O$ associates outputs with states. Extended, $\delta : Q \times (\Sigma^I)^* \rightarrow Q$, where $\delta(q, \varepsilon) = q$ and $\delta(q, \sigma \cdot s) =$

⁴ Either a pair of identical representations, or between which a translation exists.

⁵ See Definition 10 for PAC-identification and related Theorems 4 and 5.

$\delta(\delta(q, \sigma), s)$. In *Mealy machines* [32], outputs are associated with transitions instead, so $L : Q \times \Sigma^I \rightarrow \Sigma^O$. Mealy and Moore machines are equivalent [19] and can be converted between one another. Reward machines are an application of Mealy machines in reinforcement learning and are used to express a class of non-Markovian reward functions.

Active Automaton Learning. Consider the problem of actively learning a Moore machine $\langle Q, q_0, \Sigma^I, \Sigma^O, \delta, L \rangle$ to exactly model a function $f : (\Sigma^I)^* \rightarrow \Sigma^O$, where Σ^I and Σ^O are input and output alphabets of finite size known to both teacher and learner. We desire a learner which learns a model $\hat{f}$ of f exactly; that is for all $s \in (\Sigma^I)^*$, we require $\hat{f}(s) = f(s)$, where $\hat{f}(s) = L(\delta(q_0, s))$, with the assistance of a teacher $\mathcal{T}$ that can answer questions about f .

In Angluin’s seminal active learning algorithm, L^* [5], the learner learns $\hat{f}$ as a binary classifier, where $|\Sigma^O| = 2$, to determine sequence membership of a regular language by querying $\mathcal{T}$ with: (i) *membership queries*, where the learner asks $\mathcal{T}$ for the value of $f(s)$ for a particular sequence s , and (ii) *equivalence queries*, where the learner asks $\mathcal{T}$ to evaluate whether for all $s \in (\Sigma^I)^*$, $\hat{f}(s) = f(s)$. For the latter query, $\mathcal{T}$ returns True if the statement holds; otherwise a counterexample c for which $\hat{f}(c) \neq f(c)$ is returned. An *observation table* $\langle S, E, T \rangle$ records the concrete observations acquired by the learner’s queries (see Fig. 2). Here, S is a set of prefixes, E is a set of suffixes, and T is the empirical observation function that maps sequences to output values— $T : (S \cup (S \cdot \Sigma^I)) \cdot E \rightarrow \Sigma^O$. The observation table is a two-dimensional array; $s \in (S \cup (S \cdot \Sigma^I))$ indexes rows, and $e \in E$ indexes columns, with entries given by $T(s \cdot e)$. Any proposed hypothesis $\hat{f}$ must be consistent with T . The algorithm operates by construction: a deterministic transition function must be found exhibiting consistency (deterministic transitions) and operates in a closed manner over the set of states. If the consistency or closure requirements are violated, then membership queries are executed to expand the observation table. Once a suitable transition function is found, a hypothesis $\hat{f}$ can be made and checked via the equivalence query. The algorithm terminates if $\hat{f}(s) = f(s)$ for all $s \in (\Sigma^I)^*$; otherwise L^* continues on by adding counterexample c and all its prefixes to the table, and finds another transition function satisfying the consistency and closure requirements.

Consequently, we consider how L^* -style learning can be used to learn Moore machines from *preference queries* over sequences, as a foray into understanding how finite automaton structure can be learned from comparison information.

3 Problem Statement

We consider how a Moore machine $\langle Q, q_0, \Sigma^I, \Sigma^O, \delta, L \rangle$ can be actively learned from *preference queries* over $(\Sigma^I)^*$. We focus on the case of a finite sized Σ^I and Σ^O known to both teacher and learner. With a *preference query*, the learner asks the teacher $\mathcal{T}$ which of two sequences s_1 and s_2 is preferred, or if both are equally preferable. This requires a preference model, which we assume represents a total preordering over $(\Sigma^I)^*$; we also assume Σ^O is totally ordered. Thus, we consider a preference model for $\mathcal{T}$ where $f : (\Sigma^I)^* \rightarrow \Sigma^O$ is consistent with

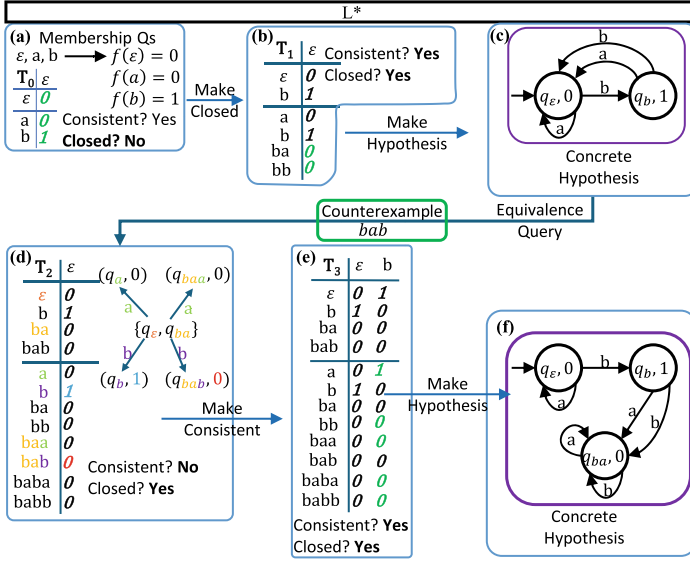


Fig. 2. L^* example. Figure 3 illustrates REMAP. L^* records concrete values from membership queries in observation table. Green symbolizes new information. Colors in (d) visually highlight transition inconsistencies.

both orderings, i.e., $\mathcal{T}$ prefers s_1 over s_2 if $f(s_1) > f(s_2)$, or otherwise has equal preference if $f(s_1) = f(s_2)$.

Several options for evaluating hypothesis equivalence (is $\hat{f} \equiv f$?) can be defined for this problem formulation. We first review the definition of exact equivalence, followed by an alternative notion of equivalence which respects ordering:

Definition 3 (Exact Equivalence). *Given a hypothesis $\hat{f}$ and a reference f , $\hat{f}$ is exactly equivalent to f if $\hat{f}(s) = f(s)$ for all s in $(\Sigma^I)^*$.*

Definition 4 (Order-Respecting Equivalence). *Given a hypothesis $\hat{f}$ and a reference f , $\hat{f}$ is order-respecting equivalent to f if, for all s, t in $(\Sigma^I)^*$, there exists a relation $\mathbf{R}_{s,t} \in \{=, >, <\}$ such that $\hat{f}(s)\mathbf{R}_{s,t}\hat{f}(t) \iff f(s)\mathbf{R}_{s,t}f(t)$.*

Since a hypothesis $\hat{f}$ satisfying exact equivalence is also always order-respecting, we focus on exact equivalence queries where $\mathcal{T}$ returns feedback along with counterexample c .

Definition 5 (Exact Equivalence Query with Feedback). *Given a hypothesis $\hat{f}$ and a reference f , an exact equivalence query EQ returns the following triple:*

$$EQ(\hat{f}) = \langle \forall s \in (\Sigma^I)^* : \hat{f}(s) = f(s), c : \exists c \text{ s.t. } \hat{f}(c) \neq f(c), \phi(c) \rangle$$

where c is a counterexample if one exists, and $\phi(c)$ is feedback associated with the counterexample, interpreted as a constraint.

The strength of feedback ϕ directly impacts how many hypotheses per counterexample can be eliminated by the learner: returning $\phi(c) := \hat{f}(c) \neq f(c)$, which means the value $f(c)$ is not $\hat{f}(c)$, is weak feedback compared to returning $\phi(c) := \hat{f}(c) = f(c)$, which means the value of $\hat{f}(c)$ should be $f(c)$, or returning $\phi(c) := \hat{f}(c) \in X$ (where $X \subset \Sigma^O$, which means the value of $\hat{f}(c)$ should be in a subset of Σ^O). Although REMAP outputs a concrete Moore machine, REMAP navigates symbolic Moore machine⁶ space using solely preference information, while concrete information assists in selecting the concrete hypothesis. Thus, our theoretical analysis considers equivalence queries that provide counterexamples with strong feedback: $\phi(c) := \hat{f}(c) = f(c)$.

4 The REMAP ALGORITHM

REMAP is a L^* -based algorithm employing preference and equivalence queries to gather constraints. By first leveraging unification to navigate the symbolic hypothesis space of Moore machines, solving the constraints yields a concrete Moore machine. In particular, REMAP (Algorithm 1) learns a Moore machine $\langle \hat{Q}, \hat{q}_0, \Sigma^I, \Sigma^O, \hat{\delta}, \hat{L} \rangle$, representing a multiclass classifier $\hat{f}(s) = \hat{L}(\hat{\delta}(\hat{q}_0, s))$.

Central to REMAP are four core components: **(1)** a new construct called a *symbolic observation table* (Sect. 4.1), shown in Fig. 3a, 3b, 3d, and 3e; **(2)** as well as an associated algorithm for *unification* [35] (Sect. 4.2) inspired by Martelli and Montanari [31] to contain the fresh variable explosion; both enable the learner to **(3)** generate symbolic hypotheses (Sect. 4.3) purely from observed symbolic constraints, along with **(4)** a constraint solver for obtaining a concrete hypothesis (Sect. 4.4). We first discuss the four components and how they fit together into REMAP (Sect. 4.5). Afterwards, we illustrate the correctness and termination guarantees.

4.1 Symbolic Observation Tables

Recall that in classic L^* , the observation table entries are concrete values obtained from membership queries (Fig. 2a). However, observations obtained from *preference queries* are constraints, rather than concrete values, indicating that for a pair of sequences s_1 and s_2 , one of $f(s_1) > f(s_2)$, $f(s_1) < f(s_2)$, or $f(s_1) = f(s_2)$ holds. We therefore introduce the *symbolic observation table* $\langle S, E, T; \mathcal{C}, \Gamma \rangle$, where S is a set of prefixes, E is a set of suffixes, $\mathcal{C}$ is the set of known constraints, Γ is a *context* which uniquely maps sequences to variables, with the set of variables $\mathcal{V}$ in the context Γ given by $\mathcal{V} = \{\Gamma[s \cdot e] \mid s \cdot e \in (S \cup (S \cdot \Sigma^I)) \cdot E\}$, and $T : (S \cup (S \cdot \Sigma^I)) \cdot E \rightarrow \mathcal{V}$ maps queried sequences to variables. Thus, in a symbolic observation table, the entry for each prefix-suffix pair

⁶ i.e., a Moore machine with symbolic values as outputs.

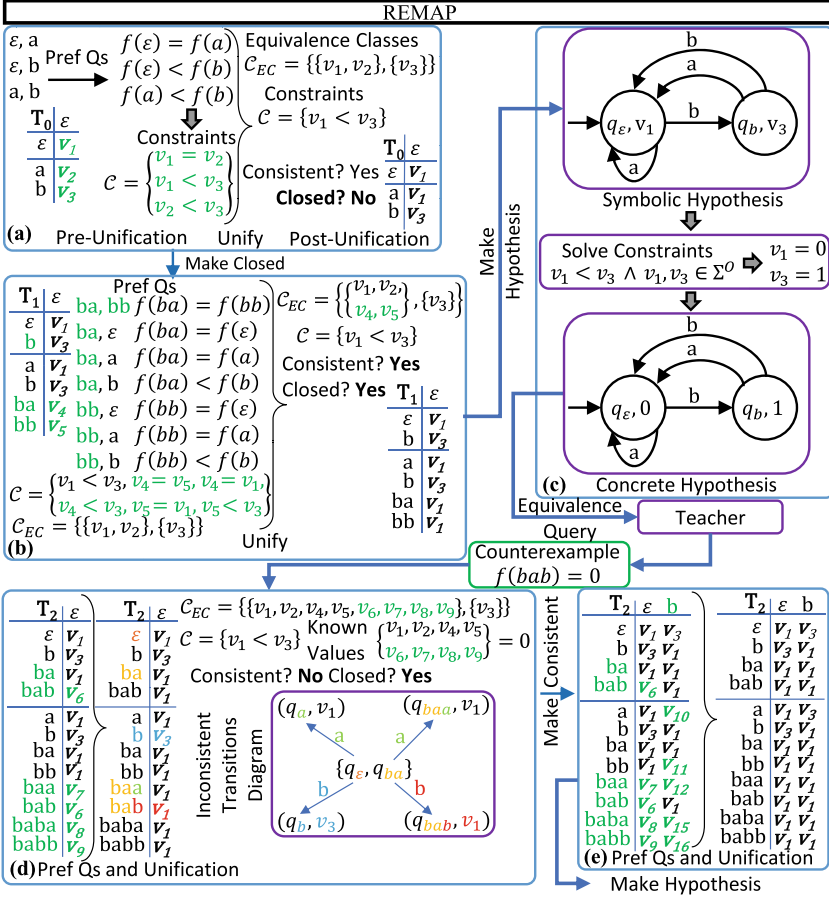


Fig. 3. REMAP example. Figure 2 illustrates L^* . REMAP performs preference queries and records variables in symbolic observation table $\langle S, E, T; C, \Gamma \rangle$ within a SYMBOLICFILL. (a) initializes $\langle S, E, T; C, \Gamma \rangle$; (b) expands S with sequence b , followed by SYMBOLICFILL; yields unified, closed, and consistent $\langle S, E, T; C, \Gamma \rangle$; (c) MAKEHYPOTHESIS yields concrete hypothesis h_1 from symbolic hypothesis and constraints in C ; (d) submits h_1 via equivalence query; receives and processes counterexample bab with feedback $f(bab) = 0$, adding bab , ba , and b to S , performs a SYMBOLICFILL, sets the value of equivalence class for $T(bab)$ to $f(bab)$; yields an inconsistent table. (e) shows resulting consistent table. Figure 2f shows ground truth Moore machine with $\Sigma^I = \{a, b\}$, $\Sigma^O = \{0, 1\}$. Green symbolizes new information. Colors in (d) visually highlight transition inconsistencies. (Color figure online)

is a variable, rather than a concrete value; and the constraints over those variables are stored in C (Fig. 3a, 3b, 3d, 3e). The constraints from the preferences of $\mathcal{T}$ over f about s_1 and s_2 correspond with $T(s_1) > T(s_2)$, $T(s_1) < T(s_2)$, or $T(s_1) = T(s_2)$, respectively.

Definition 6. An *equivalence class* $\mathbb{C}$ of variables is a set with the property that all members of $\mathbb{C}$ are equivalent to each other. The *representative* $\llbracket \mathbb{C} \rrbracket$ of $\mathbb{C}$ is a deterministically elected member of $\mathbb{C}$. The set of variables $\mathcal{V}$ can be partitioned into disjoint equivalence classes. The set of equivalence classes $\mathcal{C}_{EC}$ corresponds with the partitioning of $\mathcal{V}$ into the smallest possible number of equivalence classes consistent with equality constraints. Let $\mathcal{R} = \{\llbracket \mathbb{C} \rrbracket \mid \mathbb{C} \in \mathcal{C}_{EC}\}$ be the set of representatives.

4.2 Unification and Constraints

Since *preference queries* return observations comparing the values of two sequences, we leverage a simple unification algorithm to ensure the number of unique variables in the table remains bounded by $|\Sigma^O|$. As a reminder, unification involves symbolically solving for representatives and rewriting all constraints and variables in terms of those representatives. Whenever we observe the constraint $f(s_1) = f(s_2)$, this implies $T(s_1) = T(s_2)$, so we add $\Gamma[s_1]$ and $\Gamma[s_2]$ to an *equivalence class* $\mathbb{C}$, and elect a *representative* from $\mathbb{C}$. The unification algorithm is presented in Appendix.

If $\Gamma[s_1]$ already belongs to an equivalence class $\mathbb{C} \in \mathcal{C}_{EC}$, but $\Gamma[s_2]$ is an *orphan variable*—belonging to no class in $\mathcal{C}_{EC}$ —then $\Gamma[s_2]$ is merged into $\mathbb{C}$. Swap s_1 and s_2 for the other case. If $\Gamma[s_1]$ and $\Gamma[s_2]$ belong to separate classes $\mathbb{C}_1$ and $\mathbb{C}_2$, with $\mathbb{C}_1 \neq \mathbb{C}_2$, then $\mathbb{C}_1$ and $\mathbb{C}_2$ are merged into one via $\mathbb{C} \leftarrow \mathbb{C}_1 \cup \mathbb{C}_2$, and one of $\llbracket \mathbb{C}_1 \rrbracket$ and $\llbracket \mathbb{C}_2 \rrbracket$ is deterministically elected as the representative $\llbracket \mathbb{C} \rrbracket$.

When this unification process is applied to a symbolic observation table $\langle S, E, T; \mathcal{C}, \Gamma \rangle$, we are unifying $\mathcal{V}$ (the set of variables in the context Γ) according to the set of known equivalence constraints $\mathcal{C}_{EQ}$ and equivalence classes $\mathcal{C}_{EC}$ available in $\mathcal{C}$; and each variable in the table is replaced with its equivalence class representative (see Fig. 3abde after the large right curly brace). That is, for all s in $(S \cup S \cdot \Sigma^I) \cdot E$, we substitute $T(s) \leftarrow \llbracket \mathbb{C} \rrbracket$ if $T(s) \in \mathbb{C}$. In the resulting *unified* symbolic observation table, T maps queried sequences to the set of representatives $\mathcal{R}$. Note $|\mathcal{R}| = |\mathcal{C}_{EC}| \leq |\Sigma^O|$.

Besides constraints from preferences queries, the learner obtains constraints about the value of $T(c)$ from *equivalence queries* (Fig. 3c to 3d) and adds them to $\mathcal{C}$. When first obtained, these constraints may possibly be expressed in terms of orphan variables, but during the process of unification, each orphan variable joins an equivalence class and is then replaced by its equivalence class representative in the constraint. Thus, after unification, all constraints in $\mathcal{C}$ are expressed in terms of equivalence class representatives.

Finally, unifying the symbolic observation table is critical for making a symbolic hypothesis without knowledge of concrete value assignments: unification permits $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ to become *closed* and *consistent*—prerequisites for generating a symbolic hypothesis.

Definition 7. Let $\mathbf{rows}(S) = \{\mathbf{row}(s) \mid s \in S\}$, where the row in $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ indexed by s is $\mathbf{row}(s)$. $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is *closed* if $\mathbf{rows}(S \cdot \Sigma^I) \subseteq \mathbf{rows}(S)$.

Algorithm 1. REMAP**Input:** Alphabets Σ^I (input) and Σ^O (output), teacher $\mathcal{T}$ **Output:** Moore Machine $\mathcal{H} = \langle \hat{Q}, \Sigma^I, \Sigma^O, \hat{q}_0, \hat{\delta}, \hat{L} \rangle$

```

1: Initialize  $\mathcal{O} = \langle S, E, T; \mathcal{C}, \Gamma \rangle$  with  $S = \{\varepsilon\}, E = \{\varepsilon\}, \mathcal{C} = \{\}, \Gamma = \emptyset$ 
2:  $\mathcal{O} \leftarrow \text{SYMBOLICFILL}(\mathcal{O}|\mathcal{T})$ 
3: repeat
4:    $\mathcal{O} \leftarrow \text{MAKECLOSEDANDCONSISTENT}(\mathcal{O}|\mathcal{T})$ 
5:    $\mathcal{H} \leftarrow \text{MAKEHYPOTHESIS}(\mathcal{O}, \Sigma^I, \Sigma^O)$ 
6:    $\text{result} \leftarrow \text{EQUIVALENCEQUERY}(\mathcal{H}|\mathcal{T})$ 
7:    $\mathcal{O} \leftarrow \text{PROCESSCEX}(\mathcal{O}, \text{result})$  if  $\text{result} \neq \text{correct}$ 
8: until  $\text{result} = \text{correct}$ 
9: return  $\mathcal{H}$ 

```

Definition 8. $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is **consistent** if for all sequence pairs s_1 and s_2 where $\text{row}(s_1) \equiv \text{row}(s_2)$, all their transitions also remain equivalent with each other: $\text{row}(s_1 \cdot \sigma) \equiv \text{row}(s_2 \cdot \sigma)$ for all $\sigma \in \Sigma^I$.

Definition 9. Table $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is **unified** if $T(s \cdot e) \in \mathcal{R}$ for all $s \cdot e \in (S \cup (S \cdot \Sigma^I)) \cdot E$.

4.3 Making a Symbolic Hypothesis

If symbolic observation table $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is *unified*, *closed*, and *consistent*, a symbolic hypothesis can be made (Fig. 3c). This construction is identical to L^* , except that the outputs are symbolic:

$$\begin{aligned}
 \hat{Q} &= \{\text{row}(s) | \forall s \in S\} \text{ is the set of states} \\
 \hat{q}_0 &= \text{row}(\varepsilon) \text{ is the initial state} \\
 \hat{\delta}(\text{row}(s), \sigma) &= \text{row}(s \cdot \sigma) \text{ for all } s \in S \text{ and } \sigma \in \Sigma^I \\
 \hat{L}(\text{row}(s)) &= T(s \cdot \varepsilon) \text{ is the sequence to output function} \\
 \langle \hat{Q}, \hat{q}_0, \Sigma^I, \Sigma^O, \hat{\delta}, \hat{L} \rangle &\text{ is a symbolic hypothesis.}
 \end{aligned}$$

4.4 Making a Concrete Hypothesis

The learner finds a satisfying solution Λ to the set of constraints $\mathcal{C}$, while subject to the global constraint requiring the value of each representative to be in Σ^O (Fig. 3c). Thus, $\hat{L}$ becomes concrete.

$$\Lambda \leftarrow \text{FINDSOLUTION}(\langle S, E, T; \mathcal{C}, \Gamma \rangle, \Sigma^O) \quad (1)$$

$$\hat{L}(\text{row}(s)) = \Lambda[T(s \cdot \varepsilon)] \quad (2)$$

In particular, Λ finds satisfying values for each member of $\mathcal{R}$, the set of equivalence class representatives. Since $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is unified, we are guaranteed that T maps from queried sequences to $\mathcal{R}$; hence $\Lambda[T(s \cdot \varepsilon)]$ is guaranteed to resolve to a concrete value as long as the teacher provides consistent preferences.

Algorithm 2. A Query Efficient Symbolic Fill Procedure**procedure** SYMBOLICFILL($\langle S, E, T; \mathcal{C}, \Gamma \rangle | T$)

-
- 1: $seqs = \{\}$; Let $\mathcal{O} = \langle S, E, T; \mathcal{C}, \Gamma \rangle$; let $oldsortedseqs$ be a sorted list of sequences.
 - 2: $seqs = \text{POPULATEMISSINGFRESHVARS}(\mathcal{O})$
 - 3: $\mathcal{O} \leftarrow \text{PREFQsByRANDOMIZEDQUICKSORTFOLLOWEDBYLINEARMERGE}(seqs, oldsortedseqs, \mathcal{O} | T)$
 - 4: $\mathcal{O} \leftarrow \text{UNIFICATION}(\mathcal{O})$
 - 5: **return** $\mathcal{O}$
-

4.5 REMAP

We now describe REMAP (Algorithm 1) in terms of the previously discussed components. In order to make a symbolic hypothesis, REMAP must first obtain a *unified, closed, and consistent* $\langle S, E, T; \mathcal{C}, \Gamma \rangle$. To perform closed and consistency checks, the table must be unified. Therefore, REMAP must *symbolically fill* $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ by asking preference queries and performing unification to obtain a unified table. If the unified table is not closed or not consistent, then the table is alternately expanded and symbolically filled until the table becomes unified, closed, and consistent.

SYMBOLICFILL. A symbolic fill (Algorithm 2 and Appendix) (i) creates fresh variables for empty entries in the table, (ii) asks preference queries, and (iii) performs unification. If a sequence $s \cdot e \in (S \cup (S \cdot \Sigma^I)) \cdot E$ does not have an associated variable in the context Γ , then a fresh variable $\Gamma[s \cdot e]$ is created. Preference queries are executed to obtain the total preordering of $(S \cup (S \cdot \Sigma^I)) \cdot E$. In our implementation, we use preference queries in place of comparisons in randomized quicksort and linear merge. Once every sequence in $(S \cup (S \cdot \Sigma^I)) \cdot E$ has a variable, and once the preference queries have been completed, unification is performed. SYMBOLICFILL is called on lines 2, 4 (in MAKECLOSEDANDCONSISTENT), and 7 (in PROCESSEX.). Unification is shown in Appendix.

Ensuring Consistency. If the unified table is not consistent, then there exists a pair $s_1, s_2 \in S$ and $\sigma \in \Sigma^I$ for which $\text{row}(s_1) \equiv \text{row}(s_2)$ and $\text{row}(s_1 \cdot \sigma) \neq \text{row}(s_2 \cdot \sigma)$, implying there is an $e \in E$ such that $T(s_1 \cdot \sigma \cdot e) \neq T(s_2 \cdot \sigma \cdot e)$. To attempt to make the table consistent, add $\sigma \cdot e$ to E , and then perform a symbolic fill. Figure 3d shows inconsistency, and Fig. 3e shows a table made consistent through expansion of the suffix set.

Ensuring Closedness. If the unified table is not closed, then $\text{rows}(S \cdot \Sigma^I) \not\subseteq \text{rows}(S)$. To attempt to make the table closed, find a row $\text{row}(s')$ in $\text{rows}(S \cdot \Sigma^I)$ but not in $\text{rows}(S)$. Add s' to S , update $S \cdot \Sigma^I$, then fill symbolically. Figure 3a to 3b shows a closure process.

The closed and consistency checks occur in a loop (consistency first, closed second) inside MAKECLOSEDANDCONSISTENT until the table becomes unified, closed, and consistent. Then hypothesis $h = \langle \hat{Q}, \hat{q}_0, \Sigma^I, \Sigma^O, \hat{\delta}, \hat{L} \rangle$ is generated by

MAKEHYPOTHESIS (Fig. 3c) and is sent to the teacher via EQUIVALENCEQUERY (Fig. 3c to 3d). If h is wrong, then a counterexample c is returned, as well as feedback $\phi(c)$ which is interpreted as a new constraint on the value of $\hat{f}(c)$. The counterexample c and all its prefixes are added to S , and then a symbolic fill is performed. Then the constraint on the value of $\hat{f}(c)$ is added to $\mathcal{C}$ as a constraint on the value of the representative at $T(c)$.

5 Theoretical Guarantees of REMAP

We now cover the algorithmic guarantees of REMAP when $\mathcal{T}$ uses exact equivalence queries, and show how sampling-based equivalence queries achieves PAC-identification. We first detail how REMAP guarantees termination and yields a correct, minimal Moore machine that classifies sequence equivalently to f . If REMAP terminates, then the final hypothesis must be correct, since termination occurs only if no counterexamples exist for the final hypothesis. Therefore, if the hypothesized Moore machine classifies all sequences correctly according to the teacher, it must be correct. Thus, proving termination implies correctness. See Appendix for sketches and proofs. Here, we assume the teacher provides feedback $\hat{f}(c) = f(c)$ with counterexample c .

Theorem 1. *If $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is unified, closed, and consistent, and the range of MAKEHYPOTHESIS($\langle S, E, T; \mathcal{C}, \Gamma \rangle$) is $\mathcal{H}$, then every hypothesis $h \in \mathcal{H}$ is consistent with constraints $\mathcal{C}$. Any other hypothesis consistent with $\mathcal{C}$, but not contained in $\mathcal{H}$, must have more states.*

Proof. (Sketch) A given unified, closed, and consistent symbolic observation table $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ corresponds to $(\mathcal{S}, \mathcal{R}, \mathcal{C})$, where $\mathcal{S}$ is a symbolic hypothesis, $\mathcal{R}$ is the set of representatives used in the table, and $\mathcal{C}$ are the constraints expressed over $\mathcal{R}$. All hypotheses in $\mathcal{H}$ have states and transitions identical to $\mathcal{S}$. Each satisfying solution A to $\mathcal{C}$ corresponds to a unique concrete hypothesis in $\mathcal{H}$. Therefore every concrete hypothesis in $\mathcal{H}$ is consistent with $\mathcal{C}$. Let $|h|$ represent the number of states in h . We know for all $h \in \mathcal{H}$, $|h| = |\mathcal{S}|$. Let $\overline{\mathcal{H}}$ be the set of concrete hypotheses *not* in $\mathcal{H}$. Note $\overline{\mathcal{H}}$ can be partitioned into three sets—concrete hypotheses with (a) fewer states than $\mathcal{S}$, (b) more states than $\mathcal{S}$, and (c) same number of states as $\mathcal{S}$ but inconsistent with $\mathcal{C}$. We ignore (c) because we care only about hypotheses consistent with $\mathcal{C}$. Consider any concrete hypothesis h in $\mathcal{H}$ and its corresponding satisfying solution A . Suppose we desire another hypothesis h' to be consistent with h . If $|h'| < |h|$, then h' cannot be consistent with h because at least one sequence will be misclassified. Therefore, if h' must be consistent with h , then we require $|h'| \geq |h|$. Thus, any other hypothesis consistent with $\mathcal{C}$, but not in $\mathcal{H}$, must have more states. $\square$

Theorem 1 establishes that the output of REMAP will be the smallest Moore machine consistent with all the constraints in $\mathcal{C}$. This is necessary to prove termination.

Lemma 1. *Whenever a counterexample c is processed, either 0 or 1 additional representative values becomes known.*

Theorem 2. *Suppose $\langle S, E, T; \mathcal{C}, \Gamma \rangle$ is unified, closed, and consistent. Let $\hat{h} = \text{MAKEHYPOTHESIS}(\langle S, E, T; \mathcal{C}, \Gamma \rangle)$ be the hypothesis induced by Λ , a satisfying solution to $\mathcal{C}$. If the teacher returns a counterexample c as the result of an equivalence query on $\hat{h}$, then at least one of the following is true about $\hat{h}$: (a) $\hat{h}$ contains too few states, or (b) the satisfying solution Λ inducing $\hat{h}$ is either incomplete or incorrect.*

Corollary 1. *REMAP must terminate when the number of states and number of known representative values in a concrete hypothesis reach their respective upper bounds.*

Proof. (Sketch) This sketch applies to the above lemma, theorem, and corollary about termination. Consider the sequence $\dots, h_{k-1}, h_k, \dots$ of hypotheses that REMAP makes. For a given pair of consecutive hypotheses (h_{k-1}, h_k) , consider how the number of states n , and the number of *known* representative values $n_\bullet$ changes. Let n^* be the number of states of the minimal Moore machine correctly classifying all sequences. Let $V^* \leq |\Sigma^O|$ be the upper bound on $|\mathcal{R}|$. Note that $0 \leq n_\bullet \leq |\mathcal{R}| \leq V^* \leq |\Sigma^O|$ always holds. Through detailed case analysis on returned counterexamples, we can show that the change in $n_\bullet$, denoted by $\Delta n_\bullet$, must always be either 0 or 1, and furthermore, if $\Delta n_\bullet = 0$, then we must have $\Delta n \geq 1$. By the case analysis and tracking n and $n_\bullet$, observe that if a counterexample c is received from the teacher due to hypothesis h , then *at least one of* (a) $n < n^*$ or (b) $n_\bullet < V^*$ must be true. Since $\Delta n_\bullet$ and Δn cannot simultaneously be 0, whenever a new hypothesis is made, progress must be made towards the upper bound of (n^*, V^*) . If the upper bound is reached, then the algorithm must terminate, since it is impossible to progress from the point (n^*, V^*) . $\square$

Theorem 3 (Query Complexity). *If n is the number of states of the minimal automaton isomorphic to the target automaton, and m is the maximum length of any counterexample sequence that the teacher returns, then (a) REMAP executes at most $n + |\Sigma^O| - 1$ equivalence queries, and (b) the preference query complexity is $\mathcal{O}(mn^2 \ln(mn^2))$, which is polynomial in the number of unique sequences queried.*

Proof. Based on Theorem 2, we know that the maximum number of equivalence queries is the taxi distance from the point $(1, 0)$ to $(n, |\Sigma^O|)$, which is $n + |\Sigma^O| - 1$. From counterexample processing, we know there will be at most $m(n + |\Sigma^O| - 1)$ sequences added to the prefix set S , since a counterexample c of length m results in at most m sequences added to the prefix set S . The maximum number of times the table can be found inconsistent is at most $n - 1$ times, since there can be at most n states, and the learner starts with 1 state. Whenever a sequence is added to the suffix set E , the maximum length of sequences in E increases by at most 1, implying the maximum sequence length in E is $n - 1$. Similarly, closure

operations can be performed at most $n-1$ times, so the total number of sequences in E is at most n ; the maximum number of sequences in S is $n+m(n+|\Sigma^O|-1)$. The maximum number of unique sequences queried in the table is the maximum cardinality of $(S \cup S \cdot \Sigma^I) \cdot E$, which is

$$(n + m(n + |\Sigma^O| - 1))(1 + |\Sigma^I|)n = \mathcal{O}(mn^2).$$

Therefore, the preference query complexity of REMAP is $\mathcal{O}(mn^2 \ln(mn^2))$ due to randomized quicksort. $\square$

Lemma 1, Theorem 2, and Corollary 1 imply REMAP makes progress towards termination with every hypothesis, and termination occurs when specific conditions are satisfied; therefore its output must be correct. Theorem 3 indicates that REMAP learns the correct minimal automaton isomorphic to the target automaton in polynomial time. Next, we show how REMAP achieves PAC-identification when sampling-based equivalence queries are used.

Definition 10 (Probably Approximately Correct Identification).

Given Moore machine $M = \langle Q, q_0, \Sigma^I, \Sigma^O, \delta, L \rangle$, let the classification function $f : (\Sigma^I)^* \rightarrow \Sigma^O$ be represented by $f(s) = L(\delta(q_0, s))$ for all $s \in (\Sigma^I)^*$. Let $\mathcal{D}$ be any probability distribution over $(\Sigma^I)^*$. An algorithm $\mathcal{A}$ probably approximately correctly identifies f if and only if for any choice of $0 < \epsilon \leq 1$ and $0 < d < 1$, $\mathcal{A}$ always terminates and outputs an ϵ -approximate sequence classifier $\hat{f} : (\Sigma^I)^* \rightarrow \Sigma^O$, such that with probability at least $1 - d$, the probability of misclassification is $P(\hat{f}(s) \neq f(s)) \leq \epsilon$ when s is drawn according to the distribution $\mathcal{D}$.

Theorem 4. REMAP achieves probably approximately correct identification of any Moore machine when the teacher $\mathcal{T}$ uses sampling-based equivalence queries with at least $m_k \geq \lceil \frac{1}{\epsilon} (\ln \frac{1}{d} + k \ln 2) \rceil$ samples drawn i.i.d. from $\mathcal{D}$ for the k th equivalence query.

Proof. (Sketch) The probability $1 - \epsilon_k$ of a sequence sampled from an arbitrary distribution $\mathcal{D}$ over $(\Sigma^I)^*$ depends upon the distribution and the intersections of sets of sequences of the teacher and the learner's k th hypothesis with the same classification values. The probability that the k th hypothesis misclassifies a sequence is ϵ_k . If the teacher samples m_k samples for the k th equivalence query, then an upper bound can be established for the case when $\epsilon_k \leq \epsilon$ for a given ϵ . Since we know REMAP executes at most $n + |\Sigma^O| - 1$ equivalence queries, one can upper bound the probability that REMAP terminates with an error by summing all probabilities of events that the teacher does not detect an error in at most $n + |\Sigma^O| - 1$ equivalence queries. An exponential decaying upper bound can be found, and a lower bound for m_k can be found in terms of ϵ, d , and k . $\square$

Theorem 5. To achieve PAC-identification under REMAP, given parameters ϵ and d , and if f can be represented by a minimal Moore machine with n states and $|\Sigma^O|$ classes, then teacher $\mathcal{T}$ needs to sample at least

$$\mathcal{O} \left(n + |\Sigma^O| + \frac{1}{\epsilon} \left((n + |\Sigma^O|) \ln \frac{1}{d} + (n + |\Sigma^O|)^2 \right) \right)$$

sequences i.i.d. from $\mathcal{D}$ over the entire run of REMAP.

Proof. Since for the k th equivalence query, the teacher must sample at least $m_k \geq \lceil \frac{1}{\epsilon}(\ln \frac{1}{d} + k \ln 2) \rceil$ sequences in order to achieve PAC-identification, if the total number of samples is to be minimized while still achieving PAC-identification, then the teacher can just sample a quantity of sequences i.i.d. from $\mathcal{D}$ equal to the following total

$$\begin{aligned}
 & \sum_{k=1}^{n+|\Sigma^O|-1} \left\lceil \frac{1}{\epsilon} \left(\ln \frac{1}{d} + k \ln 2 \right) + 1 \right\rceil \\
 &= n + |\Sigma^O| - 1 + \frac{1}{\epsilon} \left[\left(\ln \frac{1}{d} \right) (n + |\Sigma^O| - 1) + \ln 2 \sum_{k=1}^{n+|\Sigma^O|-1} k \right] \\
 &= \mathcal{O} \left((n + |\Sigma^O|) + \frac{1}{\epsilon} \left((n + |\Sigma^O|) \ln \frac{1}{d} + (n + |\Sigma^O|)^2 \right) \right)
 \end{aligned}$$

□

Theorem 4 and Theorem 5 imply REMAP achieves PAC-identification for a choice of ϵ and d as long as the teacher samples sufficient sequences per equivalence query. In particular, Theorem 5 indicates the total quantity of sequences sampled by the teacher to achieve PAC-identification depends on both n the number of states, and $|\Sigma^O|$ the number of output classes. This contrasts with the result for PAC-identification of DFAs (Theorem 7 [5]), which depends only on n . Finally, since REMAP outputs a Moore machine, one can leverage Moore and Mealy machine equivalence in order to convert the final hypothesis into a reward machine, as defined and covered in the next section.

6 Learning Reward Machines from Preferences

We consider applying REMAP to learn reward machines from preferences. Reward machines are Mealy machines with propositional and reward semantics. Equivalence between Mealy and Moore machines allows the output of REMAP to be converted to a reward machine. We first review reinforcement learning and reward machine semantics.

Markov Decision Processes. Decision making problems are often modeled by a Markov Decision Process (MDP), which is a tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ represents the transition probability from state s to s' via action a . The reward function $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ provides the associated scalar reward, and $0 \leq \gamma \leq 1$ is a discount factor. In MDPs, the Markovian assumption is that transitions and rewards depend only upon the current state-action pair and the next state. However, not all tasks are expressible using Markovian reward [2].

Non-markovian Reward. Non-Markovian Reward Decision Processes are identical to MDPs, except that $R : (\mathcal{S} \times \mathcal{A})^* \rightarrow \mathbb{R}$ is non-Markovian a reward

function that depends on state-action history. This allows reward machines to model a class of non-Markovian reward functions.

Reward Machines. A reward machine (RM) is a Mealy machine where $\Sigma^I = 2^{\mathcal{P}}$, Σ^O is a set of reward emitting objects, and $\mathcal{P}$ is a set of propositions describing states and actions. A *labeling* function $\mathbb{L} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow 2^{\mathcal{P}}$ with $\mathbb{L}(s_{k-1}, a_k, s_k) = l_k$ labels a state-action sequence $s_0 a_1 s_1 a_2 s_2 \dots a_n s_n$ with label sequence $l_1 l_2 \dots l_n$. Thus, reward machines operate over label sequences.

A single disjunctive normal formula (DNF) labeled transition can summarize multiple transitions with identical Σ^O labels (connecting a pair of states), since the elements of $2^{\mathcal{P}}$ are sets of propositions. Reward machines map label sequences to reward outputs and can be represented as $f : (2^{\mathcal{P}})^* \rightarrow \Sigma^O$, so REMAP learns a reward machine by converting the output Moore machine to a reward machine.

Sequential Tasks in OfficeWorld and CraftWorld. Icarte et al. [25] and Andreas et al. [4] introduced the OfficeWorld and CraftWorld gridworld domains, respectively, and feature sequential tasks encoded as reward machines. OfficeWorld features 4 sequential tasks across several rooms with various objects available for an agent to interact with. Example tasks include (1) picking up coffee and mail and delivering them to a certain room, or (2) continuously patrolling between a set of rooms. CraftWorld is a 2D version of Minecraft, where the 10 sequential tasks involve the agent collecting materials and constructing tools or objects in a certain order while avoiding hazards.

7 Empirical Results

We evaluate the exact and PAC-identification (PAC-ID) versions of REMAP and consider: first, *how often is PAC-ID REMAP correct?* (Exact REMAP is guaranteed to be correct). To answer this, we run experiments by applying PAC-ID REMAP to learn reward machines (RMs), by converting the Moore machine into a RM. We measure empirical correctness with *empirical probability of isomorphism* and *average regret*. Second, *how do exact and PAC-ID REMAP scale?* We measure preference query complexity as a function of the number of unique sequences queried, and present an example phase diagram of algorithm execution. We use Z3 [15] for the constraint solver.

Setup. We investigate these questions on 14 sequential tasks in the OfficeWorld and CraftWorld domains. The Appendix contains domain specific details. We implement exact equivalence queries (Definition 5) using a variant of the Hopcroft-Karp algorithm [3, 24]. We implement i.i.d. sequence sampling in sampling-based equivalence queries with the following process per sample: sample a length L from a geometric distribution; then, construct an L -length sequence by drawing L elements i.i.d. from a uniform distribution over Σ^I . In both the exact and sampling-based equivalence queries, strong feedback is used.

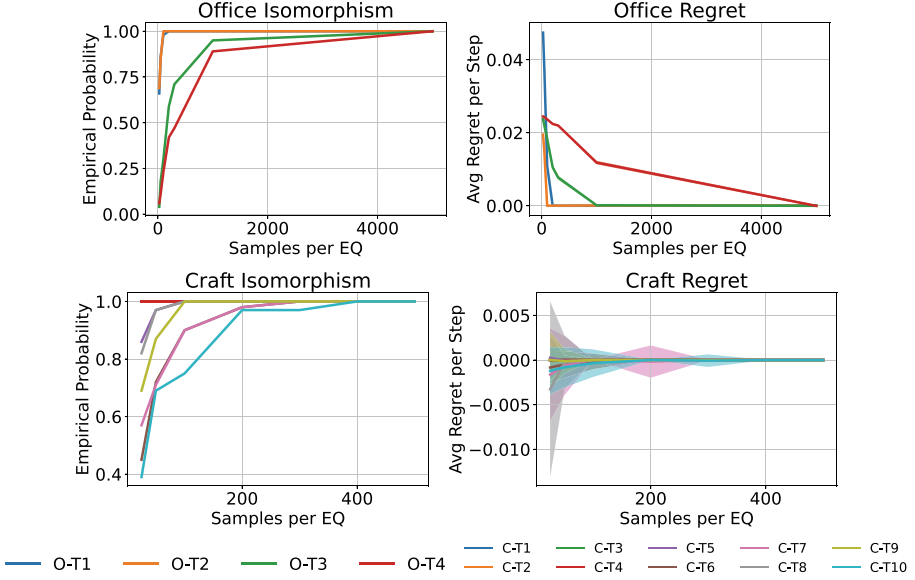


Fig. 4. PAC-identification REMAP: (left) empirical isomorphism probability, (right) average regret as functions of the number of samples per equivalence query for 4 Office-World tasks (O-T1-4) and 10 CraftWorld tasks (C-T1-10).

7.1 PAC-Identification Correctness Experiments

Reproducibility. PAC-ID REMAP was run 100 times per ground truth reward machine. We measure correctness based on (1) empirical probability that the learned RM is isomorphic to the ground truth RM, based on classification accuracy, and (2) average policy regret between the learned RM policy and the ground truth RM policy.

Empirical Probability of Isomorphism is the fraction of learned RMs with 100% classification accuracy. As the number of sample sequences tested by the teacher per equivalence query increases, the probability that the learner outputs a RM isomorphic to the ground truth RM upon termination goes to 1 (Fig. 4, left column). Classification accuracy is defined as the fraction of a test set of sequences that are identically classified by the learned and ground truth RMs. The Appendix describes the distribution over $(\Sigma^I)^*$ that the test set is drawn from.

Average Policy Regret. We employ *Q-learning with counterfactual experiences for reward machines* (CRM) [41] to obtain optimal policies for ground truth and learned RMs. We measured the empirical expected return of optimal policies learned from each type of RM. Average regret for a given task was measured as the difference between the empirical return under the ground truth RM for that task (averaged over 100 CRM trials) and the empirical return under the learned RM (with 10 CRM trials per learned RM, then averaging all

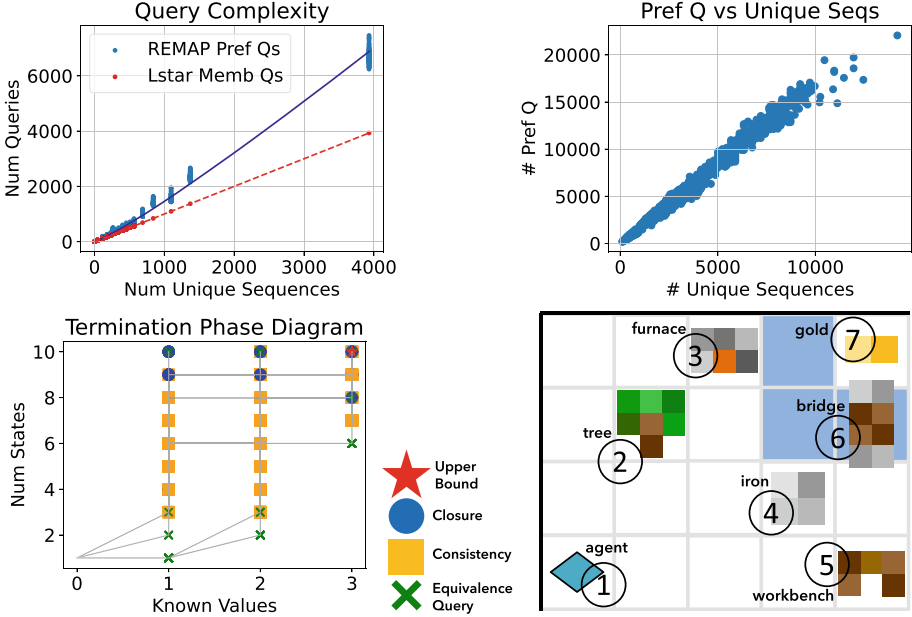


Fig. 5. Query Complexity. *Top left:* Exact REMAP preference query complexity. Mean of 100 trials per ground truth reward machine (blue dots) ± 1 standard deviation (orange, grey bars). *Top right:* PAC-ID REMAP, preference query complexity is $\mathcal{O}(n \ln n)$ in the number of unique sequences in the table. *Bottom left:* Example termination phase diagram. *Bottom right:* CraftWorld environment depiction. (Color figure online)

$100 \times 10 = 1000$ trials). Regret goes to 0 as the number of samples tested by the teacher per equivalence query increases (Fig. 4, right column). The Appendix describes regret computation details.

Correctness Conclusion. Exact REMAP learns the correct automaton 100% of the time. Additionally, PAC-ID REMAP is more likely to be correct as the number of samples per equivalence query increases: isomorphism probability goes to 1 and regret goes to 0 for all tasks in both domains.

7.2 Scaling Experiments

Figure 5 shows query complexity results. We measure preference query complexity of exact and PAC-ID REMAP, as a function of the number of unique sequences stored in the table upon termination. Exact REMAP (upper left) displays a trendline of $C = 0.2114N \ln N$ with $R^2 = 0.99268$, where C is the number of queries, and N is the number of unique sequences in the observation table. PAC-ID REMAP (upper right) tends to make significantly more preference queries about unique sequences compared to exact REMAP due to the sampling process. However, the number of preference queries is still $\mathcal{O}(N \ln N)$ due to

randomized quicksort and linear merge comparison complexity. In comparison, an L^* -based approach would use exactly N membership queries (exactly linear in N with a coefficient of 1).

The maximum number of equivalence queries REMAP makes (Theorem 3) is the taxi distance from $(0, 1)$ to $(|\Sigma^O|, n)$ in the termination phase diagram of Fig. 5. Progress (Lemma 1 and Theorem 2) towards termination (Corollary 1) occurs whenever a new hypothesis is made. REMAP can terminate early when all variables have correct values and the required number of states is reached.

8 Related Work

Active approaches for learning automata are variations or improvements of Angluin’s seminal L^* algorithm [5], featuring *membership* and *equivalence queries*. We consider an alternative formulation: *actively learning automata from preference and equivalence queries* featuring feedback. We first discuss adaptations of L^* for learning variants of finite automata, including reward machine variants.

Learning Finite Automata. Angluin [5] introduced L^* to learn deterministic finite automata (DFAs). REMAP has similar theoretical guarantees as L^* , but utilizes a symbolic observation table, rather than an evidence-based one. Other algorithms adopt the evidence-based table of L^* to learn: symbolic automata [7, 17], where Boolean predicates summarize state transitions; weighted automata [8, 9] which feature valuation semantics for sequences on non-deterministic automata; probabilistic DFAs [44], a weighted automata that models distributions of sequences. None of these approaches uses preference queries.

However, Shah et al. [38] considers active, cost-based selection between membership and preference queries to learn DFAs, relying on a satisfiability encoding of the problem. They assume a *fixed hypothesis space* and have *probabilistic* guarantees for termination and correctness. REMAP, through unification, *navigates a sequence of hypothesis spaces*, each guaranteed to contain a concrete hypothesis satisfying current constraints, and has theoretical guarantees of correctness, minimalism, and termination under *exact* and *PAC-identification* settings.

Furthermore, learning finite automata from preference information relates to the novel problem of *learning reward machines* [25] *from preferences*. Learning Markovian reward functions from preferences has been studied extensively using neural [11, 36] and interpretable decision tree [10, 27] representations, but approaches for learning reward machines primarily adapt evidence-based finite automata learning approaches.

Reward Machine Variants. Several reward machine (RM) variants have been proposed. Classical RMs [25, 42] have deterministic transitions and rewards; probabilistic RMs [16] model probabilistic transitions and deterministic rewards; and stochastic RMs [14] pair deterministic transitions with stochastic rewards. Symbolic RMs [47] are deterministic like classical RMs, but feature symbolic reward values in place of concrete values. Zhou and Li [47] apply Bayesian

inverse reinforcement learning (BIRL) to infer optimal reward values and actualize symbolic RMs into classical RMs, and require a symbolic RM sketch. REMAP requires no sketch, since it navigates over a hypothesis space of symbolic RMs and outputs a concrete classical RM upon termination.

Learning Reward Machines. Many RM learning algorithms assume access to explicit reward samples via environment interaction. Given a maximum RM size, Toro Icarte et al. [42] apply discrete optimization to arrive at a perfect classical RM. Xu et al. [45] learn a minimal classical RM by combining regular positive negative inference [18] with Q-learning for RMs [25], and apply constraint solving to ensure each hypothesis RM is consistent with observed reward samples. Corazza et al. [14] extended the method to learn stochastic RMs. Topper et al. [40] extends BIRL to learn classical RMs using simulated annealing, but needs the number of states to be supplied, and requires empirical tuning of hyperparameters. L^* based approaches have also been used to learn classical [20, 39, 46] and probabilistic [16] RMs, relying on concrete observation tables. Gaon and Brafman [20] and Xu et al. [46] use a binary observation table, while Tappler et al. [39] and Dohmen et al. [16] record empirical reward distribution table entries.

In contrast, REMAP uses a symbolic observation table, and uses preferences information in place of explicit reward values. REMAP navigates symbolic hypothesis space, with constraint solving enabling a concrete classical RM.

9 Conclusion

We introduce the problem of learning Moore machines from preferences and propose REMAP, an L^* based algorithm, wherein a strong learner with access to a constraint solver is paired with a weak teacher capable of answering preference queries and providing counterexample feedback in the form of a constraint. Unification applied to a symbolic observation table permits symbolic hypothesis space navigation; the constraint solver enables concrete hypotheses. REMAP has theoretical guarantees for correctness, termination, and minimalism under both exact and PAC-identification settings, and it has been empirically verified under both settings when applied to learning reward machines. Future work will expound on more realistic preference models, variable strength feedback, and inconsistency.

Acknowledgments. This work is supported in part by the National Science Foundation (DGE-2125858, IIS-2416461), the Defense Advanced Research Projects Agency (HR00112320018, HR00112490431), and the Army Research Office (W911NF2110009). Any opinions, findings, and conclusions expressed in this material are those of the authors and do not necessarily reflect the views of the sponsors.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aarts, F., Kuppens, H., Tretmans, J., Vaandrager, F.W., Verwer, S.: Learning and testing the bounded retransmission protocol. In: International Conference on Graphics and Interaction (2012). <https://api.semanticscholar.org/CorpusID:2641499>
2. Abel, D., et al.: On the expressivity of Markov reward. In: Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J.W. (eds.) *Advances in Neural Information Processing Systems*, vol. 34, pp. 7799–7812, Curran Associates, Inc. (2021). https://proceedings.neurips.cc/paper_files/paper/2021/file/4079016d940210b4ae9ae7d41c4a2065-Paper.pdf
3. Almeida, M., Moreira, N., Reis, R.: Testing the equivalence of regular languages. In: *Workshop on Descriptive Complexity of Formal Systems* (2009). <https://api.semanticscholar.org/CorpusID:9014414>
4. Andreas, J., Klein, D., Levine, S.: Modular multitask reinforcement learning with policy sketches. In: Precup, D., Teh, Y.W. (eds.) *Proceedings of the 34th International Conference on Machine Learning, Proceedings of Machine Learning Research*, vol. 70, pp. 166–175, PMLR (2017). <https://proceedings.mlr.press/v70/andreas17a.html>
5. Angluin, D.: Learning regular sets from queries and counterexamples. *Inf. Comput.* **75**(2), 87–106 (1987). [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6)
6. Angluin, D.: Queries and concept learning. *Mach. Learn.* **2**, 319–342 (1988)
7. Argyros, G., D’antoni, L.: The learnability of symbolic automata. In: *International Conference on Computer Aided Verification* (2018)
8. Balle, B., Mohri, M.: Learning weighted automata. In: *Conference on Algebraic Informatics* (2015)
9. Bergadano, F., Varricchio, S.: Learning behaviors of automata from multiplicity and equivalence queries. *SIAM J. Comput.* **25**, 1268–1280 (1994)
10. Bewley, T., Lécué, F.: Interpretable preference-based reinforcement learning with tree-structured reward functions. [arXiv:abs/2112.11230](https://arxiv.org/abs/2112.11230) (2021). <https://api.semanticscholar.org/CorpusID:245353680>
11. Bıyık, E., Talati, A., Sadigh, D.: APReL: A library for active preference-based reward learning algorithms (2022)
12. Christiano, P.F., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D.: Deep reinforcement learning from human preferences. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc. (2017). https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf
13. Christiano, P.F., Leike, J., Brown, T.B., Martic, M., Legg, S., Amodei, D.: Deep reinforcement learning from human preferences. [arXiv:abs/1706.03741](https://arxiv.org/abs/1706.03741) (2017). <https://api.semanticscholar.org/CorpusID:4787508>
14. Corazza, J., Gavran, I., Neider, D.: Reinforcement learning with stochastic reward machines. In: *Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022*, pp. 6429–6436, AAAI Press (2022). <https://ojs.aaai.org/index.php/AAAI/article/view/20594>

15. De Moura, L., Bjørner, N.: Z3: an efficient SMT solver. In: Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, pp. 337–340, TACAS’08/ETAPS’08, Springer-Verlag, Berlin, Heidelberg (2008). ISBN 3540787992
16. Dohmen, T., Topper, N., Atia, G.K., Beckus, A., Trivedi, A., Velasquez, A.: Inferring probabilistic reward machines from non-markovian reward signals for reinforcement learning. In: Kumar, A., Thiébaux, S., Varakantham, P., Yeoh, W. (eds.) Proceedings of the Thirty-Second International Conference on Automated Planning and Scheduling, ICAPS 2022, Singapore (virtual), June 13–24, 2022, pp. 574–582, AAAI Press (2022). <https://ojs.aaai.org/index.php/ICAPS/article/view/19844>
17. Drews, S., D’antoni, L.: Learning symbolic automata. In: International Conference on Tools and Algorithms for Construction and Analysis of Systems (2017)
18. Dupont, P.: Regular grammatical inference from positive and negative samples by genetic search: the GIG method. In: Carrasco, R.C., Oncina, J. (eds.) Grammatical Inference and Applications, Second International Colloquium, ICGI-94, Alicante, Spain, September 21–23, 1994, Proceedings, Lecture Notes in Computer Science, vol. 862, pp. 236–245, Springer (1994), https://doi.org/10.1007/3-540-58473-0_152
19. Fleischner, H.: On the equivalence of mealy-type and Moore-type automata and a relation between reducibility and Moore-reducibility. *J. Comput. Syst. Sci.* **14**(1), 1–16 (1977). ISSN 0022-0000, [https://doi.org/10.1016/S0022-0000\(77\)80038-X](https://doi.org/10.1016/S0022-0000(77)80038-X), URL <https://www.sciencedirect.com/science/article/pii/S002200007780038X>
20. Gaon, M., Brafman, R.I.: Reinforcement learning with non-markovian rewards. In: The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7–12, 2020, pp. 3980–3987, AAAI Press (2020)
21. Giannakopoulou, D., Rakamaric, Z., Raman, V.: Symbolic learning of component interfaces. In: Sensors Applications Symposium (2012). <https://api.semanticscholar.org/CorpusID:1449946>
22. Gold, E.M.: Complexity of automaton identification from given data. *Inf. Control.* **37**, 302–320 (1978). <https://api.semanticscholar.org/CorpusID:8943792>
23. Guerin, J.T., Allen, T.E., Goldsmith, J.: Learning CP-net preferences online from user queries. In: AAAI Conference on Artificial Intelligence (2013). <https://api.semanticscholar.org/CorpusID:15976671>
24. Hopcroft, J.E., Karp, R.M.: A linear algorithm for testing equivalence of finite automata. (1971). <https://api.semanticscholar.org/CorpusID:120207847>
25. Icarte, R.T., Klassen, T., Valenzano, R., McIlraith, S.: Using reward machines for high-level task specification and decomposition in reinforcement learning. In: Dy, J., Krause, A. (eds.) Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research, vol. 80, pp. 2107–2116, PMLR (2018). <https://proceedings.mlr.press/v80/icarte18a.html>
26. Iyengar, S.S., Lepper, M.R.: When choice is demotivating: can one desire too much of a good thing? *J. Pers. Soc. Psychol.* **79**(6), 995 (2000)
27. Kalra, A., Brown, D.S.: Can differentiable decision trees learn interpretable reward functions? [arXiv:abs/2306.13004](https://arxiv.org/abs/2306.13004) (2023). <https://api.semanticscholar.org/CorpusID:259224487>

28. Koriche, F., Zanuttini, B.: Learning conditional preference networks with queries. *Artif. Intell.* **174**, 685–703 (2009). <https://api.semanticscholar.org/CorpusID:3060370>
29. Lin, S.W., Étienne André, Liu, Y., Sun, J., Dong, J.S.: Learning assumptions for compositional verification of timed systems. *IEEE Trans. Softw. Eng.* **40**(2), 137–153 (2014)
30. MacGlashan, J., et al.: Interactive learning from policy-dependent human feedback. In: Precup, D., Teh, Y.W. (eds.) *Proceedings of the 34th International Conference on Machine Learning, Proceedings of Machine Learning Research*, vol. 70, pp. 2285–2294, PMLR (2017). <https://proceedings.mlr.press/v70/macglashan17a.html>
31. Martelli, A., Montanari, U.: Unification in linear time and space: a structured presentation. *Tech. rep.*, Istituto di Elaborazione della Informazione, Pisa (1976)
32. Mealy, G.H.: A method for synthesizing sequential circuits. *Bell Syst. Tech. J.* **34**(5), 1045–1079 (1955). <https://doi.org/10.1002/j.1538-7305.1955.tb03788.x>
33. Moore, E.F.: Gedanken-experiments on sequential machines. In: Shannon, C., McCarthy, J. (eds.) *Automata Studies*, pp. 129–153. Princeton University Press, Princeton, NJ (1956)
34. Ouyang, L., et al.: Training language models to follow instructions with human feedback. [arXiv:abs/2203.02155](https://arxiv.org/abs/2203.02155) (2022). <https://api.semanticscholar.org/CorpusID:246426909>
35. Robinson, J.A.: A machine-oriented logic based on the resolution principle. *J. ACM* **12**, 23–41 (1965)
36. Sadigh, D., Dragan, A.D., Sastry, S., Seshia, S.A.: Active preference-based learning of reward functions. In: *Robotics: Science and Systems* (2017)
37. Schuts, M., Hooman, J., Vaandrager, F.: Refactoring of Legacy Software Using Model Learning and Equivalence Checking: An Industrial Experience Report. Springer International Publishing (2016)
38. Shah, A., Vazquez-Chanlatte, M., Junges, S., Seshia, S.A.: Learning formal specifications from membership and preference queries (2023)
39. Tappler, M., Aichernig, B.K., Bacci, G., Eichlseder, M., Larsen, K.G.: L*-based learning of Markov decision processes. In: *International Symposium on Formal Methods*, pp. 651–669, Springer (2019)
40. Topper, N., Velasquez, A., Atia, G.: Bayesian inverse reinforcement learning for non-markovian rewards (2024)
41. Toro Icarte, R., Klassen, T.Q., Valenzano, R.A., McIlraith, S.A.: Reward machines: exploiting reward function structure in reinforcement learning. *J. Artif. Intell. Res. (JAIR)* **73**, 173–208 (2022). <https://doi.org/10.1613/jair.1.12440>
42. Toro Icarte, R., Waldie, E., Klassen, T., Valenzano, R., Castro, M., McIlraith, S.: Learning reward machines for partially observable reinforcement learning. In: Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc. (2019). <https://proceedings.neurips.cc/paper/2019/file/532435c44bec236b471a47a88d63513d-Paper.pdf>
43. Valiant, L.G.: A theory of the learnable. *Commun. ACM* **27**(11), 1134–1142 (1984). ISSN 0001-0782, <https://doi.org/10.1145/1968.1972>
44. Weiss, G., Goldberg, Y., Yahav, E.: Learning deterministic weighted automata with queries and counterexamples. In: Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc. (2019). https://proceedings.neurips.cc/paper_files/paper/2019/file/d3f93e7766e8e1b7ef66dfdd9a8be93b-Paper.pdf

45. Xu, Z., Gavran, I., Ahmad, Y., Majumdar, R., Neider, D., Topcu, U., Wu, B.: Joint inference of reward machines and policies for reinforcement learning. *Proc. Int. Conf. Autom. Plann. Sched.* **30**(1), 590–598 (2020). <https://doi.org/10.1609/icaps.v30i1.6756>, <https://ojs.aaai.org/index.php/ICAPS/article/view/6756>
46. Xu, Z., Wu, B., Ojha, A., Neider, D., Topcu, U.: Active finite reward automaton inference and reinforcement learning using queries and counterexamples. In: *Machine Learning and Knowledge Extraction: 5th IFIP TC 5, TC 12, WG 8.4, WG 8.9, WG 12.9 International Cross-Domain Conference, CD-MAKE 2021, Virtual Event, August 17–20, 2021, Proceedings*, pp. 115–135, Springer-Verlag, Berlin, Heidelberg (2021). ISBN 978-3-030-84059-4, https://doi.org/10.1007/978-3-030-84060-0_8
47. Zhou, W., Li, W.: A hierarchical Bayesian approach to inverse reinforcement learning with symbolic reward machines. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., Sabato, S. (eds.) *Proceedings of the 39th International Conference on Machine Learning, Proceedings of Machine Learning Research*, vol. 162, pp. 27159–27178, PMLR (2022). <https://proceedings.mlr.press/v162/zhou22b.html>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

